

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

简单动态规划：从一维到二维

Introduction to Dynamic Programming

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia
2026 年 10 月 14 日

目录

1. 动态规划入门
2. 一维 DP 经典题
3. 二维 DP 经典题
4. 练习题
5. 总结



目录

1 动态规划入门

2 一维 DP 经典题

3 二维 DP 经典题

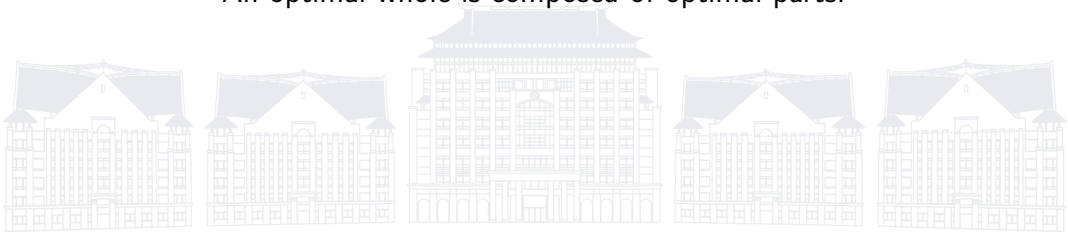
4 练习题

5 总结

引言

The past cannot be affected by the present, but the future can be.

An optimal whole is composed of optimal parts.



什么是动态规划

动态规划（Dynamic Programming, DP）：把一个大问题拆成若干个**相互重叠**的子问题，先解决子问题并**保存结果**，再用子问题的答案拼出原问题的答案。

- **最优子结构**：大问题的最优解，由子问题的最优解拼成
- **重叠子问题**：不同分支反复用到同一个子问题
- **无后效性**：当前状态确定后，后续决策只与当前状态有关

$$\text{DP} = \text{状态定义} + \text{转移方程} + \text{边界条件}$$

DP 和递归的区别

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

朴素递归

- 从原问题往下拆
- 同一个子问题被算很多次
- 复杂度常常是指数级
- 例：斐波那契 $O(2^n)$

动态规划

- 从最小子问题往上推
- 每个子问题只算一次
- 结果存进数组，随时复用
- 斐波那契优化到 $O(n)$

DP 的本质：用空间换时间，把重复计算变成查表

DP 的解题四步法

1. **定义状态**：$dp[\ldots]$ 表示什么？这是最关键的一步
2. **找转移方程**：$dp[i]$ 由哪些更小的状态推出
3. **确定边界**：最小规模时 dp 的值是多少
4. **确定顺序**：从小规模往大规模推，保证用到的状态已经算好

状态定义错了，后面全错；状态定义对了，方程往往一目了然

一维 DP 和二维 DP

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

一维 DP

- 状态只用一个下标
- 形式：$dp[i]$
- 典型：爬楼梯、打家劫舍
- 答案往往就是 $dp[n]$

二维 DP

- 状态需要两个下标
- 形式：$dp[i][j]$
- 典型：网格、最长公共子序列
- 答案在 $dp[n][m]$ 或某一行/列的最值

先从一维入门，理解套路后再上二维

目录

1 动态规划入门

2 一维 DP 经典题

3 二维 DP 经典题

4 练习题

5 总结

例题一：爬楼梯

题目：有 n 级台阶，每次可以走 1 级或 2 级，问走到第 n 级共有多少种走法。

示例

- $n = 1$ 时，答案为 1（直接走 1 级）
- $n = 2$ 时，答案为 2（1+1，或 2）
- $n = 3$ 时，答案为 3（1+1+1，1+2，2+1）

爬楼梯：解法

四步法

- 状态：$dp[i]$ 表示走到第 i 级的走法数
- 转移：最后一步只能走 1 级或 2 级，$dp[i] = dp[i-1] + dp[i-2]$
- 边界：$dp[1] = 1$，$dp[2] = 2$
- 顺序：i 从 3 到 n 依次递推

为什么这样转移

- 走到第 i 级的最后一步：要么从 $i-1$ 级走 1 级来，要么从 $i-2$ 级走 2 级来
- 这两类走法不重不漏，加起来就是总走法数

$$\text{当前的走法数} = \text{上一级走法数} + \text{上两级走法数}$$

爬楼梯：代码


```cpp
// dp[i] 表示走到第 i 级的走法数
int climbStairs(int n) {
    if (n <= 2) return n;
    vector<int> dp(n + 1, 0);
    dp[1] = 1;
    dp[2] = 2;
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];   // 转移方程 *
    }
    return dp[n];
}

// 空间优化：只需保留前两级
int climbStairsOpt(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;                // a = dp[i-2], b = dp[i-1]
    for (int i = 3; i <= n; i++) {
        int c = a + b;
```

爬楼梯：代码

```
19          a = b;
20          b = c;
21      }
22      return b;
23  }
```

例题二：打家劫舍

题目：一条街上有 n 户人家，第 i 户有 $a[i]$ 元。不能偷相邻的两户，问最多能偷多少钱。

示例

- $a = [1, 2, 3, 1]$，答案为 4（偷第 1 户和第 3 户）
- $a = [2, 7, 9, 3, 1]$，答案为 12（偷第 1、3、5 户）

打家劫舍：解法

四步法

- 状态：$dp[i]$ 表示前 i 户能偷到的最大金额
- 转移：第 i 户要么不偷，要么偷
- 方程：$dp[i] = \max(dp[i-1],\ dp[i-2] + a[i])$
- 边界：$dp[0] = 0$，$dp[1] = a[1]$

两种选择

- 不偷第 i 户：答案等于前 $i-1$ 户的最优解 $dp[i-1]$
- 偷第 i 户：第 $i-1$ 户不能偷，等于 $dp[i-2] + a[i]$

每一步在“偷”和“不偷”里取较大者

打家劫舍：代码


```cpp
// dp[i] 表示前 i 户能偷到的最大金额
int rob(vector<int>& a) {
    int n = a.size();
    if (n == 0) return 0;
    if (n == 1) return a[0];
    vector<int> dp(n + 1, 0);
    dp[0] = 0;
    dp[1] = a[0];
    for (int i = 2; i <= n; i++) {
        // 不偷第 i 户，或偷第 i 户并加上前 i-2 户的最优解 *
        dp[i] = max(dp[i-1], dp[i-2] + a[i-1]);
    }
    return dp[n];
}

// 空间优化：只保留前两个状态
int robOpt(vector<int>& a) {
    int prev = 0, cur = 0;
```

打家劫舍：代码


```
19      for (int x : a) {
20          int next = max(cur, prev + x);
21          prev = cur;
22          cur = next;
23      }
24      return cur;
25  }
```

例题三：最大子数组和

题目：给定数组 $a[1..n]$，找一个连续子数组，使其元素之和最大，输出这个最大和。

示例

- $a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$
- 最大和子数组是 $[4, -1, 2, 1]$，和为 6

最大子数组和：解法

四步法

- 状态：$dp[i]$ 表示以 $a[i]$ 结尾的最大子数组和
- 转移：要么接在 $a[i-1]$ 后面，要么从 $a[i]$ 重新开始
- 方程：$dp[i] = \max(dp[i-1] + a[i],\ a[i])$
- 答案：$\max_i dp[i]$，而不是 $dp[n]$

关键点

- 状态必须“以 i 结尾”，才能保证转移的连续性
- 如果子数组必须包含 $a[i]$，那么只能从 $dp[i-1]$ 接上或重开
- 最终答案不一定是 $dp[n]$，要对所有 $dp[i]$ 取最大值

“以 i 结尾”是一个非常重要的一维 DP 状态套路

最大子数组和：代码


```cpp
// dp[i] 表示以 a[i] 结尾的最大子数组和
int maxSubArray(vector<int>& a) {
    int n = a.size();
    vector<int> dp(n, 0);
    dp[0] = a[0];
    int ans = dp[0];
    for (int i = 1; i < n; i++) {
        dp[i] = max(dp[i-1] + a[i], a[i]);   // 接上或重开 *
        ans = max(ans, dp[i]);
    }
    return ans;
}

// 空间优化：滚动变量
int maxSubArrayOpt(vector<int>& a) {
    int cur = a[0], ans = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        cur = max(cur + a[i], a[i]);
```

最大子数组和：代码


```
19          ans = max(ans, cur);
20      }
21      return ans;
22  }
```

例题四：最长上升子序列

题目：给定一个长度为 n 的数组 a，求它的最长上升子序列长度。

上升子序列：从数组中选出一组数，保持原来的相对顺序，且严格递增。

示例

- $a = [1, 7, 3, 5, 9, 4, 8]$
- 最长上升子序列是 $[1, 3, 5, 8]$，长度为 4

最长上升子序列：解法

四步法（$O(n^2)$ 写法）

- 状态：$dp[i]$ 表示**以** $a[i]$ **结尾**的最长上升子序列长度
- 转移：枚举所有 $j < i$，若 $a[j] < a[i]$，就可以接在 $dp[j]$ 后面
- 方程：$dp[i] = \max(dp[j] + 1)$，对所有满足 $a[j] < a[i]$ 的 j
- 边界：$dp[i] = 1$，每个数本身就是一个长度 1 的上升子序列

答案

- 需要对所有 $dp[i]$ 取最大值，因为最长子序列不一定以最后一个数结尾

“以 i 结尾” + 枚举前驱 j，是一维 DP 的经典套路

最长上升子序列：代码


```cpp
// dp[i] 表示以 a[i] 结尾的最长上升子序列长度
int lis(vector<int>& a) {
    int n = a.size();
    if (n == 0) return 0;
    vector<int> dp(n, 1);                  // 每个数本身长度至少为 1 *
    int ans = 1;
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i])
                dp[i] = max(dp[i], dp[j] + 1);   // 接在 a[j] 后面 *
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}
```

复杂度 $O(n^2)$；后续进阶可用二分优化到 $O(n \log n)$。

一维 DP 小结

共同套路

- 状态常是“前 i 个”或“以 i 结尾”
- 转移依赖前 1 到 n 个状态
- 边界通常是最小规模的情形

四道题对比

- 爬楼梯：只依赖前 2 项
- 打家劫舍：取 max
- 最大子数组：以 i 结尾
- 上升子序列：枚举前驱 j

一维 DP 的关键：想清楚 $dp[i]$ 到底表示什么

目录

1 动态规划入门

2 一维 DP 经典题

3 二维 DP 经典题

4 练习题

5 总结

例题五：网格路径数

题目：$n \times m$ 的网格，从左上角 $(0,0)$ 走到右下角 $(n-1, m-1)$，每次只能向右或向下走一步，问有多少种走法。

示例（3×3 网格）

- 共有 6 条路径
- 例如：右 $\rightarrow$ 右 $\rightarrow$ 下 $\rightarrow$ 下，或下 $\rightarrow$ 右 $\rightarrow$ 右 $\rightarrow$ 下等

网格路径数：解法

四步法

- 状态：$dp[i][j]$ 表示从起点走到 (i, j) 的路径数
- 转移：只能从上方或左方过来
- 方程：$dp[i][j] = dp[i-1][j] + dp[i][j-1]$
- 边界：第一行、第一列全为 1（只有一条路径）

关键点

- 第一行只能一路向右走，方案数恒为 1
- 第一列只能一路向下走，方案数恒为 1
- 其余格子，方案数是“上方方案数 + 左方方案数”

当前格子的方案数 = 上方格子方案数 + 左方格子方案数

网格路径数：代码

```cpp
// dp[i][j] 表示走到 (i, j) 的路径数
int uniquePaths(int n, int m) {
    vector<vector<int>> dp(n, vector<int>(m, 0));

    // 边界：第一行、第一列都只有一种走法
    for (int i = 0; i < n; i++) dp[i][0] = 1;
    for (int j = 0; j < m; j++) dp[0][j] = 1;

    for (int i = 1; i < n; i++)
        for (int j = 1; j < m; j++)
            dp[i][j] = dp[i-1][j] + dp[i][j-1];   // 从上方或左方来 *

    return dp[n-1][m-1];
}
```

带障碍的变体：障碍格子的 dp 直接置 0，其余转移不变。

例题六：最长公共子序列

题目：给定两个字符串 A 和 B，求它们最长的公共子序列长度。子序列不要求连续，但要保持相对顺序。

示例

- $A =$ abcde，$B =$ ace
- 最长公共子序列是 ace，长度为 3

对比提醒

- 子串：必须连续
- 子序列：可以跳过字符，只要顺序保持一致

最长公共子序列：解法

四步法

- 状态：$dp[i][j]$ 表示 A 前 i 个字符与 B 前 j 个字符的 LCS 长度
- 转移：看 $A[i]$ 与 $B[j]$ 是否相等
- 若相等：$dp[i][j] = dp[i-1][j-1] + 1$
- 否则：$dp[i][j] = \max(dp[i-1][j],\ dp[i][j-1])$
- 边界：$dp[0][j] = dp[i][0] = 0$

为什么这样转移

- 最后一个字符相同：直接接在之前的 LCS 后面
- 最后一个字符不同：至少有一个不参与匹配，取较长的分支

字符相同就“对齐加一”；不同就让较长的一边继续延伸

最长公共子序列：代码


```cpp
// dp[i][j] 表示 A 前 i 个字符与 B 前 j 个字符的 LCS 长度
int lcs(string& A, string& B) {
    int n = A.size(), m = B.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (A[i-1] == B[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;   // 字符相同，对齐加一 *
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);   // 否则取较长一边 *
        }
    }
    return dp[n][m];
}
```

答案在 $dp[n][m]$，正好是两串全长的 LCS 长度。

二维 DP 小结

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

共同套路

- 状态由两个维度共同确定
- 转移依赖左、上、左上三个方向
- 边界通常是第一行和第一列

两道题对比

- 网格路径：单纯加法
- 最长公共子序列：判断加一或取 max

画一张 dp 表格，把“每一步从哪里来”标出来，转移方程就清楚了

目录

1 动态规划入门

2 一维 DP 经典题

3 二维 DP 经典题

4 练习题

5 总结



练习一：数字三角形

题目：给定一个数字三角形，从顶部出发，每次可以走到下一行的相邻两个位置之一，一直走到最底层，求路径上所有数字之和的最大值。

示例

$$
\begin{array}{ccccccccc}
 & & & & 7 & & & & \\
 & & & 3 & & 8 & & & \\
 & & 8 & & 1 & & 0 & & \\
 & 2 & & 7 & & 4 & & 4 & \\
4 & & 5 & & 2 & & 6 & & 5
\end{array}
$$

最优路径：$7 \to 3 \to 8 \to 7 \to 5$，和为 30。

数字三角形：解法与代码

四步法

- **状态**：$dp[i][j]$ 表示从顶部走到 (i, j) 的最大路径和
- **转移**：(i, j) 只能从 $(i-1, j-1)$ 或 $(i-1, j)$ 来
- **方程**：$dp[i][j] = \max(dp[i-1][j-1],\ dp[i-1][j]) + a[i][j]$
- **边界**：$dp[0][0] = a[0][0]$，越界位置视为 $-\infty$

```cpp
// dp[i][j] 表示从顶部走到 (i, j) 的最大路径和
int maxTriangle(vector<vector<int>>& a) {
    int n = a.size();
    vector<vector<int>> dp = a;
    for (int i = 1; i < n; i++)
        for (int j = 0; j <= i; j++) {
            if (j == 0)
                dp[i][j] = dp[i-1][j] + a[i][j];          // 只能从上方来
            else if (j == i)
                dp[i][j] = dp[i-1][j-1] + a[i][j];        // 只能从左上来
```

数字三角形：解法与代码


```
11              else
12                  dp[i][j] = max(dp[i-1][j-1], dp[i-1][j]) + a[i][j];   // 取较大者
13          }
14      int ans = dp[n-1][0];
15      for (int j = 1; j < n; j++) ans = max(ans, dp[n-1][j]);
16      return ans;
17  }
```

练习二：编辑距离

题目：给定两个字符串 A 和 B，每次可以**插入**、**删除**或**替换**一个字符，求把 A 变成 B 的最少操作次数。

示例

- $A = \text{horse}$，$B = \text{ros}$
- **最少操作**：3 次
- **一种方案**：horse $\rightarrow$ rorse（替换 h 为 r）$\rightarrow$ rose（删除 r）$\rightarrow$ ros（删除 e）

编辑距离：解法

四步法

- **状态**：$dp[i][j]$ 表示 A 前 i 个字符变成 B 前 j 个字符的最少操作次数
- **若** $A[i] = B[j]$：$dp[i][j] = dp[i-1][j-1]$（**无需操作**）
- **否则取三种操作的最小值**：
  - **替换**：$dp[i-1][j-1] + 1$
  - **删除**：$dp[i-1][j] + 1$
  - **插入**：$dp[i][j-1] + 1$
- **边界**：$dp[i][0] = i$，$dp[0][j] = j$

一次操作影响一个方向，三种操作对应三个方向取最小值

编辑距离：代码


```cpp
// dp[i][j] 表示 A 前 i 个字符变成 B 前 j 个字符的最少操作次数
int editDistance(string& A, string& B) {
    int n = A.size(), m = B.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 0; i <= n; i++) dp[i][0] = i;   // 删除 i 个字符 *
    for (int j = 0; j <= m; j++) dp[0][j] = j;   // 插入 j 个字符 *
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (A[i-1] == B[j-1])
                dp[i][j] = dp[i-1][j-1];         // 字符相同，无需操作 *
            else
                dp[i][j] = 1 + min({dp[i-1][j-1],  // 替换 *
                                    dp[i-1][j],     // 删除 *
                                    dp[i][j-1]});   // 插入 *
        }
    }
    return dp[n][m];
}
```

练习三：硬币兑换

题目：给定一组硬币面值 $coins$ 和一个目标金额 $amount$，每种硬币可以使用**无限次**，求凑出金额 $amount$ 所需的**最少硬币数**。如果凑不出，返回 -1。

示例

- $coins = [1, 2, 5]$，$amount = 11$，答案为 3（5+5+1）
- $coins = [2]$，$amount = 3$，答案为 -1（凑不出）

硬币兑换：解法与代码

四步法

- 状态：$dp[j]$ 表示凑出金额 j 所需的最少硬币数
- 转移：对每种硬币 x，$dp[j] = \min(dp[j],\ dp[j-x]+1)$
- 边界：$dp[0] = 0$，其余初始化为一个极大值
- 顺序：金额**正序**遍历，因为每个硬币可以用无限次

```
1  // dp[j] 表示凑出金额 j 所需的最少硬币数
2  int coinChange(vector<int>& coins, int amount) {
3      const int INF = 1e9;
4      vector<int> dp(amount + 1, INF);
5      dp[0] = 0;
6      for (int x : coins) {
7          for (int j = x; j <= amount; j++) {        // 正序遍历，硬币可重复使用 *
8              dp[j] = min(dp[j], dp[j - x] + 1);
9          }
10     }
```

硬币兑换：解法与代码


```
11      return dp[amount] >= INF ? -1 : dp[amount];
12  }
```

与 0/1 背包对比：这里金额正序，允许同一硬币反复取

目录

1 动态规划入门

2 一维 DP 经典题

3 二维 DP 经典题

4 练习题

5 总结



DP 解题四步法回顾

1. **定义状态**：$dp[\ldots]$ 到底代表什么？
2. **转移方程**：当前状态由哪些前驱状态推出？
3. **边界条件**：最小规模时，dp 的值是多少？
4. **递推顺序**：保证用到的状态都已经被算过

状态定义是 DP 的灵魂；写方程之前，先在心里把“表格”画出来

经典题型速查

题型	状态	转移
爬楼梯	$dp[i]$：走到第 i 级	$dp[i-1]+dp[i-2]$
打家劫舍	$dp[i]$：前 i 户最优	$\max(dp[i-1], dp[i-2]+a[i])$
最大子数组	$dp[i]$：以 i 结尾	$\max(dp[i-1]+a[i], a[i])$
最长上升子序列	$dp[i]$：以 i 结尾	枚举 $j<i$ 且 $a[j]<a[i]$
网格路径	$dp[i][j]$：走到 (i,j)	上 + 左
最长公共子序列	$dp[i][j]$：两串前 i,j	相等加一，否则取 max
编辑距离	$dp[i][j]$：两串前 i,j	相同不加，否则三种操作取 min
硬币兑换	$dp[j]$：金额 j 最优	$\min(dp[j], dp[j-x]+1)$

先把这些题的代码敲熟，再去看更复杂的变形

下一步学什么

- 掌握 DP 的四步法，能用状态定义推导转移方程
- 熟练写出今天所有例题的代码，并会做空间优化
- 为后续学习更复杂的 DP 打基础：
  - 区间 DP、树形 DP、状态压缩 DP
  - 完全背包、多重背包、分组背包
  - 数位 DP、概率 DP

入门 DP 不在难题，而在把“状态定义”变成习惯

附录：常见错误

- 状态定义含糊：写方程时不知道 $dp[i]$ 到底代表什么
- 边界忘记初始化：$dp[0]$、第一行、第一列没写
- 递推顺序错误：用到了还没算好的状态
- 数组越界：下标从 1 开始时数组要多开一格
- 最长上升子序列：忘记对所有 $dp[i]$ 取最大值
- 编辑距离：漏掉三种操作中的某一种
- 硬币兑换：把正序写成倒序，导致每个硬币只能用一次

附录：空间优化速查

场景	优化方式	注意点
只依赖 $dp[i-1]$ 与 $dp[i-2]$	滚动变量	先更新旧值再更新新值
只依赖上一行	一维数组	0/1 背包必须倒序
完全背包（可重复取）	一维数组	金额必须正序
依赖 $dp[i-1][j-1]$ 与 $dp[i-1][j]$	一维数组 + 临时变量	提前把被覆盖的值存起来

空间优化是锦上添花；先写对，再优化。

附录：状态设计套路

状态形式	适用场景
$dp[i]$：前 i 个的最优解	打家劫舍、0/1 背包
$dp[i]$：以 i 结尾的最优解	最大子数组、最长上升子序列
$dp[i][j]$：两维前缀	最长公共子序列、编辑距离
$dp[i][j]$：网格坐标	网格路径数、数字三角形
$dp[j]$：容量 / 金额	背包问题、硬币兑换

先判断题目属于哪一类，再套用对应的状态形式